

UNIVERSIDAD DE ANTIOQUIA
DEPARTAMENTO DE INGENIERIA ELECTRÓNICA Y DE TELECOMUNICACIONES
INFORMATICA II



Informe Proyecto Final

Esteban García Lopez

1. Introducción

Debido al aprendizaje otorgado en el curso de informática II; para afianzar estos conocimientos se planteó crear un videojuego sencillo en dónde se puedan aplicar todos los conocimientos visto, (POO, memoria dinámica, Qt). La motivación de este proyecto es poder demostrar que ya se tiene un conocimiento mas que básico de c++, ademas de un entendimiento básico de Qt.

2. Planteamiento del problema

Se desea desarrollar un videojuego en QT, que este implementado con una arquitectura limpia utilizando la programación orientada a objetos, añadir tres movimiento físicos, y utilizar la memoria dinámica en el core del videojuego, mínimo debe tener dos niveles y poder seleccionar la dificultad para uno de los niveles.

3. Objetivos

3.1. Objetivo General

Demostrar la capacidad para el desarrollo de aplicaciones no sencillas con el manejo de c++, Qt, y la memoria dinamica, así la capacidad y disciplina para aprender por cuenta propia lo que se requiera desarrollar.

3.2. Objetivo especifico

Utilizar programación orientada a objetos, memoria dinámica, STL y Qt.

4. Especificación de requerimientos

4.1. Requisitos funcionales

- Implementación en Qt.
- Memoria dinámica.
- Programación orientada a objetos.
- Tres físicas, que no sean movimiento rectilíneo uniforme.
- Herencia propia.
- Dos niveles.
- Diagrama de clases de la capa lógica.
- Diagrama de clases de la Ui.
- Noción de dificultad.
- Música y efectos de sonido.

4.2. Requisitos no funcionales

- Sprites con una resolución muy buena.
- Música en todos los niveles.

4.3. Restricciones

- Librerías externas a Qt o c++.
- Delegación de responsabilidades a código externo.
- Manejo de la memoria dinámica, (que sea eficiente).

5. Diseño y arquitectura del sistema

5.1. Explicación jerárquica de los componentes

Jerárquicamente el componente Game es el padre de todos, este es quien genera todos los demás componentes, aquí se crea lo que es un character quien hereda a tres componentes más, "Player, Enemy, Boss", en estos tres se contiene lo que es el Projectile, quien a su vez utiliza PhysicsSystem, para implementar las físicas del proyecto, y también verificar las interacciones que tiene aquel que genere el proyectil.

5.2. Diagrama de clases

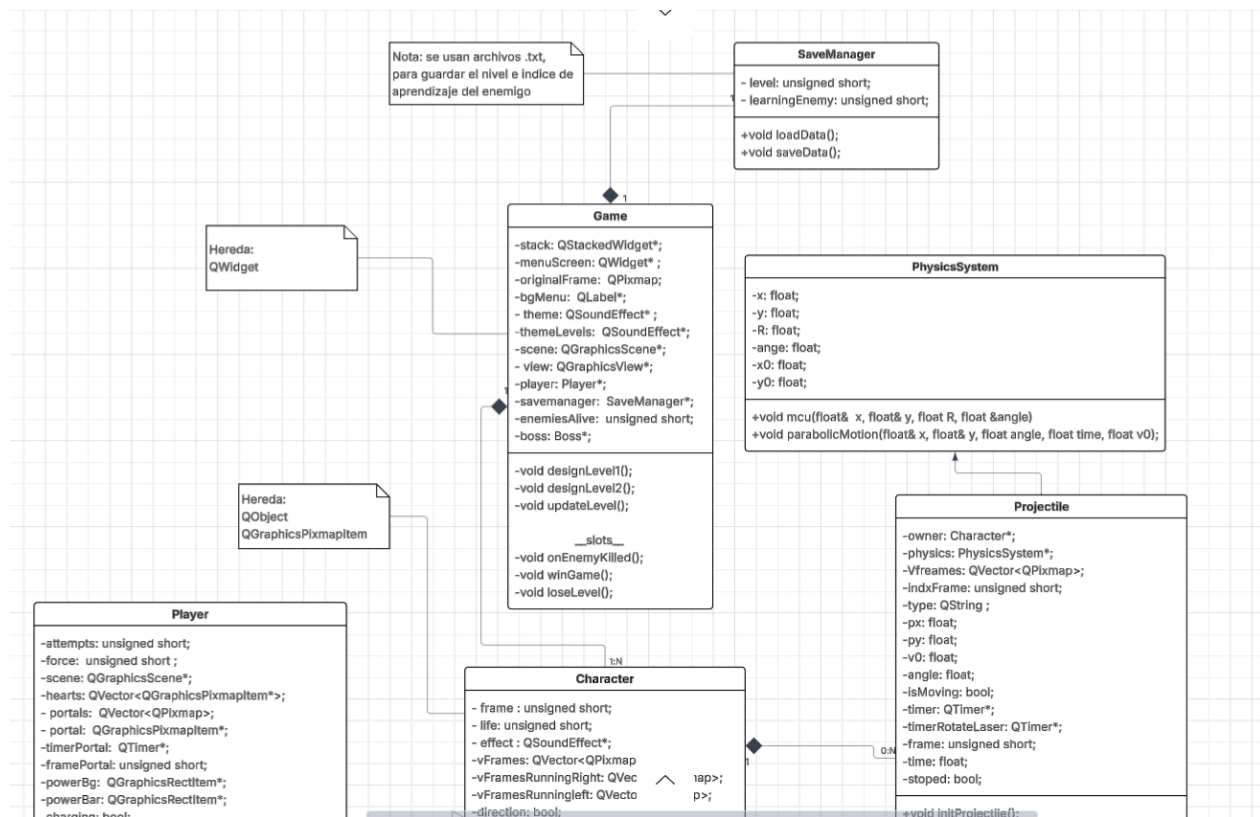


Figura 1: Diagrama de clases parte I (Archivo adjunto en el repositorio)

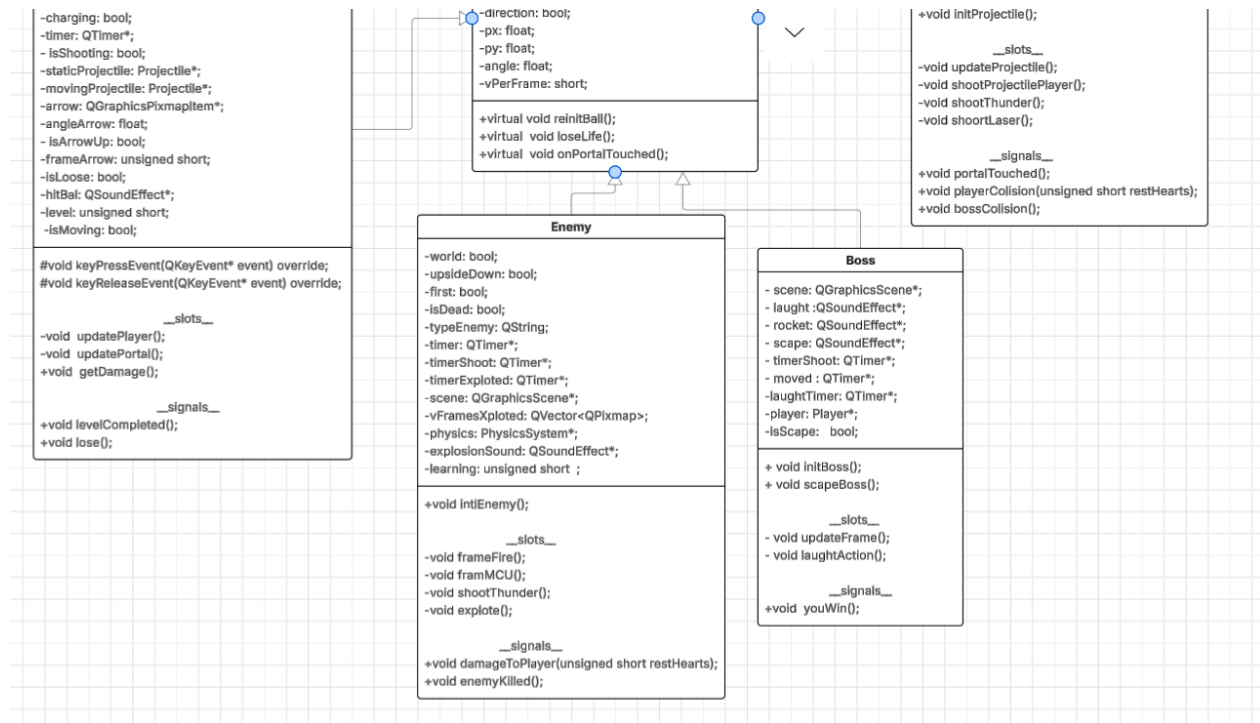


Figura 2: Diagrama de clases parte II (Archivo adjunto en el repositorio)

- Game: Es el Qwidget padre, quien se encarga de generar los demás componentes y diseñar niveles.
- Player: Es quien se encarga de generar al personaje del jugador y volverlo interactuante.
- Enemy: Es quien se encarga de generar diferentes tipos de enemigos para atacar o hacerle el nivel interesante.
- Boss: Es el componente encargado de generar un jefe de nivel y sus acciones.
- Physicsystem: Este componente brinda las físicas necesarias para que el proyectil se mueva adecuadamente.
- Projectile: Se encarga de generar proyectil, orden dada por Player, Enemy o Boss.
- saveManager: Se encarga de guardar datos del nivel y dificultad.
- Character: Funciona como padre para Player, Enemy, Boss, heredando acciones y atributos.

6. Desarrollo e implementación

Básicamente la lógica interna es sencilla, Game se encarga de generar todo lo necesario, genera al jugador, a los enemigos y al jefe, una vez generados cada uno de estos empieza a ejecutar su lógica interna, que consiste en disparar proyectiles, generar sonidos, y verificaciones de colisiones; Cuando se genera una colisión al jugador se emiten señales para desacoplar las clases, esa señal llega a la lógica del jugador en donde este ejecuta

acciones de restar vida, cambiar nivel (.^{em}ite señal al Game”), o terminar el juego. Las físicas se tratan como un componente aparte ya que esta en donde se necesiten físicas.

Los principales algoritmos son los de las físicas tales como el movimiento circular uniforme, el movimiento parabólico y la caída libre, además de los algoritmos encargados de la verificación de las colisiones para ejecutar determinadas acciones.

En cuanto a las decisiones técnicas, la más crucial que hizo cambiar parte del código, fue el de utilizar la librería QSoundEffect, y la razón es que esta utiliza audio con formato .wav, el cual su latencia es mucho más baja que el mp3 debido a que no está comprimido, perfecto para juegos.

7. Resultados y discusión

El funcionamiento del videojuego se logró sin fallas conocidas, es sencillo el videojuego pero se trabajó mucho en él y se hizo un capie en buenas prácticas.

En cuanto a las limitaciones, existen varias, la primera es la clara limitación al desconocer el desarrollo de un videojuego, manejo de imágenes o edición de las mismas, las cuales son cruciales para este tipo de juegos; Además como se aconsejaba utilizar sprites que ya se encontraran en la web, muchas veces había que conformarse con lo que se encontrara, generando entornos no muy agradables a la vista.

En las dificultades solo recalco dos, la primera es el poco conocimiento de Qt, el cual agregaba una carga alta en el desarrollo, y la segunda es que el proyecto se desarrolló individualmente debido a situaciones ajenas.

8. Conclusión

Después de la ardua dedicación y aprendizaje, siento que pude crear un juego sencillo pero que demuestra el aprendizaje en todo el curso y la interiorización de los conocimientos necesarios para hacer, porque hacer un juego por más sencillo que sea no es algo fácil, y se agradece el haberme hecho entrar en este mundo de la programación de videojuegos.

9. Referencias

- **The Qt Company.** (s.f.). *QGraphicsScene Class*. Qt Documentation.
<https://doc.qt.io/qt-6/qgraphicsscene.html>
- **OpenAI.** (2026). *ChatGPT* (Versión de lenguaje a gran escala).
<https://chatgpt.com/>
- **Google.** (2026). *Gemini* (Modelo de inteligencia artificial).
<https://gemini.google.com/app?hl=es>
- **DuarteCorporation Tutoriales.** (16 de agosto de 2014). *Tutorial Qt Creator - Introducción* [Video]. YouTube.
<https://www.youtube.com/watch?v=4TEED3VFBf&list=PL54fdmMKYUJvn4dAvziRopztp47tBRNum>

10. Anexos

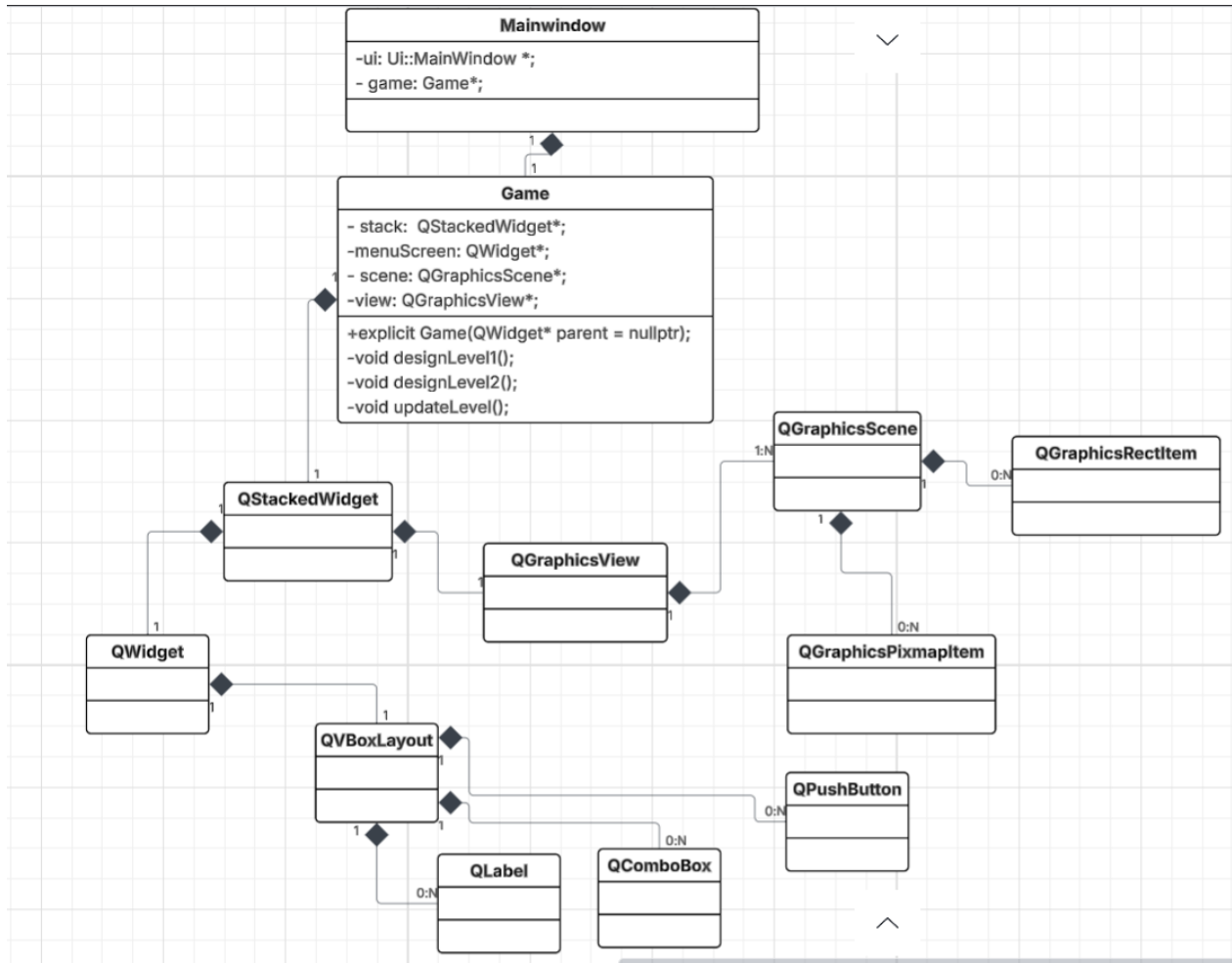


Figura 3: Diagrama de clases UI (Archivo adjunto en el repositorio)